CORPORACION UNIVERSITARIA MINUTO DE DIOS — UNIMINUTO

ESCUELA DE INGENIERIA • PROGRAMA DE INGENIERIA DE SISTEMAS

ARQUITECTURA DE DATOS — TALLER PRACTICO NoSQL & DISTRIBUIDO

Informe Tecnico: Arquitectura Cloud Native (Caso 8)

Caso 8 — Cloud Native: Plataforma SaaS Multicliente para 5.000 Organizaciones

Caso de Estudio	Caso 8 — Cloud Native: Plataforma SaaS Multicliente (5.000 Organizaciones)
Base de Datos Distribuida	`saasCloudDb` (PostgreSQL 16 + Citus 12.1 + PgBouncer + Redis Cluster)
Orquestacion e IaC	Kubernetes Multi-AZ (StatefulSets, HPA, PDB, NetworkPolicy) + Docker + Terraform
Estandar de Codigo	Nomenclatura `camelCase` (SyntheticData.py, ClusterEngine.py, TestSuite.py)
Resultado de Pruebas	8 / 8 Pruebas Ejecutadas y Verificadas (100% PASSED)

Informe Técnico: Arquitectura de Datos Cloud Native (`saasCloudDb` — Caso 8)

Programa: Ingeniería de Sistemas — Arquitectura de Datos (Taller Práctico NoSQL / Distribuido)

Caso de Estudio: Caso 8 — Cloud Native: Plataforma SaaS Multicliente (5.000 Organizaciones)

Base de Datos Distribuida: `saasCloudDb` (Nomenclatura estandarizada en **camelCase**)

A. Resumen Ejecutivo y Supuestos

A.1 Resumen Ejecutivo

Una empresa ofrece software de gestión a **5.000 organizaciones** con tamaños y patrones de uso diferentes. La plataforma debe incorporar nuevos clientes sin interrupciones (*Zero-Downtime Onboarding*), aislar los datos entre organizaciones, evitar que un cliente grande degrade a los demás (*Noisy Neighbor*), operar en varias zonas de disponibilidad y coordinar almacenamiento persistente y reemplazo de nodos.

Siguiendo la directriz del caso ("*Cloud Native describe la arquitectura y la operación. Selecciona y justifica por separado el modelo de datos*"), la solución `saasCloudDb` implementa:

- **Modelo de Datos Explícito Híbrido (Relacional Distribuido + JSONB):** Basado en **PostgreSQL 16 + Citus 12.1** (`sql/DataModel.sql`), donde las tablas (`tenants` , `tenantUsers` , `projects` , `workOrders` , `invoices` , `auditEvents`) se particionan horizontalmente (*Hash Sharding*) y se **co-localizan por** `tenantId` , mientras el campo `customAttributes JSONB` con índice GIN brinda extensibilidad por organización sin bloqueos DDL.
- **Estrategia Híbrida de Aislamiento Multi-Tenant (Pool + Silo):**
- **Modelo Pool (97 % de organizaciones — Planes `FREE` y `PRO`):** Tablas distribuidas compartidas en `nodeAz1Pool` y `nodeAz2Pool` , con aislamiento garantizado en el motor mediante **Row-Level Security (`FORCE ROW LEVEL SECURITY`)** sobre `app.currentTenantId` .
- **Modelo Silo (3 % de organizaciones — Plan `ENTERPRISE` / *Tenants Ballena*):** Aislamiento en shards exclusivos sobre el nodo dedicado `nodeAz3Silo` mediante `isolateEnterpriseTenantOnline` / `isolateWhaleTenant` .
- **Componentes Complementarios Distribuidos:**
- `pgBouncerPooler` **(Transaction Pooling):** Multiplexa 5.000 conexiones de aplicación en ~80 conexiones físicas reales hacia los nodos workers.
- `redisRateLimiter` **(3 Nodos Multi-AZ):** Aplica control de tasa por `tenantId` (*Token Bucket*) y caché de ruteo en `< 1 ms` .

A.2 Hechos del Caso vs. Supuestos Técnicos Explícitos

Tipo	Descripción	Valor / Parámetro	Justificación Técnica
Hecho (Enunciado)	Total de organizaciones clientes	**5.000 organizaciones**	Dato explícito del Caso 8
Hecho (Enunciado)	Asimetría de clientes	Tamaños y patrones de uso diferentes	Obliga a combinar Pool y Silo y mitigar *Noisy Neighbor*
Hecho (Enunciado)	Continuidad en alta de clientes	Incorporación sin interrupciones	`provisionTenant` aprovisiona en `< 25 ms` sin bloqueos globales
Supuesto 1	Distribución por Tiers (Ley de Pareto)	**75 % FREE (3.750)**, **22 % PRO (1.100)**, **3 % ENTERPRISE (150)**	Reflejado en `data/TenantsCatalog5000.csv`
Supuesto 2	Carga agregada en pico	**12.000 lecturas/s** y **2.500 escrituras/s** (80/20 Read/Write)	Calculado sobre 150 orgs Enterprise + 400 orgs Pro simultáneas
Supuesto 3	Crecimiento anual de almacenamiento	**~1,2 KB / orden**; **~2,8 TB / año**	Dimensionado con 6 TB SSD `gp3` y `allowVolumeExpansion: true`

B. Requisitos Funcionales y No Funcionales Medibles

B.1 Requisitos Funcionales (RF) y Reglas del Negocio (RN)

- **RF-01 (`provisionTenant`):** Incorporar una nueva organización, su usuario `OWNER` y su evento `TENANT_PROVISIONED` en `auditEvents` sin interrumpir a las demás organizaciones.
- **RF-02 (`createProject`, `createWorkOrder`):** Gestionar proyectos (`projects`), órdenes de trabajo (`workOrders`) y facturas (`invoices`) co-localizadas por `tenantId`.
- **RF-03 (`customAttributes JSONB`):** Soportar atributos personalizados por organización (`costCenter`, `slaHours`, `compliance`) indexados con GIN.
- **RN-01 (Invariante de Aislamiento RLS):** Ninguna organización puede leer ni alterar filas cuyo `tenantId` difiera de `app.currentTenantId`.
- **RN-02 (Invariante Financiera `chkBudgetNotExceeded`):** En todo proyecto se cumple `budgetConsumed <= budgetAllocated`.
- **RN-03 (Invariante de Idempotencia):** La restricción `UNIQUE ("tenantId", "idempotencyKey")` impide duplicar órdenes o descontar dos veces el presupuesto ante reintentos de red.

B.2 Requisitos No Funcionales Medibles (SLOs)

Métrica / Objetivo	Objetivo en Producción (Kubernetes Multi-AZ)	Resultado Medido en Prototipo Local (`TestSuite.py`)
Latencia Lectura Dashboard (`queryTenantDashboard`)	`p95 <= 15 ms`, `p99 <= 35 ms`	**`4,05 ms - 4,22 ms`** (`TEST 05` y Demo)
Tiempo de Onboarding (`provisionTenant`)	`<= 100 ms` con `0` errores en tráfico vecino	**`23,14 ms`** promedio (`12,00 ms` en Demo) con **`0` errores**
Disponibilidad Mensual del Servicio (SLA)	**99,95 %** (Multi-AZ en 3 zonas)	Failover automático verificado sin caída de servicio (`TEST 07`)
RPO (Recovery Point Objective)	**0 s** ante fallo de nodo/AZ; **<= 60 s** en desastre regional	**0 s** verificado (`1.711 / 1.711` órdenes recuperadas en `TEST 07` y `TEST 08`)
RTO (Recovery Time Objective)	**< 30 s** ante fallo de nodo/AZ; **<= 15 min** restauración PITR	**`176,17 ms`** en failover de nodo y **`27,03 ms`** en restauración de snapshot

---

C. Evaluación de Alternativas Tecnológicas y Adecuación (Requisito 8)

Criterio Técnico	Alternativa 1 (ELEGIDA): **PostgreSQL 16 + Citus 12.1 (`saasCloudDb`)**	Alternativa 2: **CockroachDB v24.1**
Arquitectura de Distribución	Coordinador distribuido + Nodos Workers con *Hash Sharding* y **Co-localización explícita** (`colocate_with`) [Citus 12.1 Docs].	Clúster simétrico dividido en *Ranges* de 512 MB replicados por consenso Raft [CockroachDB v24.1 Docs].
Row-Level Security (RLS)	**100 % nativo y maduro** (`ENABLE/FORCE ROW LEVEL SECURITY`).	Soporte limitado frente al RLS nativo de PostgreSQL 16.
Aislamiento de *Tenants Ballena* (Pool -> Silo)	Funciones nativas `isolate_tenant_to_new_shard()` y `citus_move_shard_placement()` en caliente.	Configuración por zonas menos directa para aislar un inquilino individual en caliente.
Latencia en Transacciones Single-Tenant	**Muy baja (`< 10 ms`)**: el `COMMIT` ocurre en el shard local sin 2PC entre múltiples nodos de cómputo.	Mayor latencia base (`15-40 ms`) por consenso Raft en cada escritura.
Componentes Complementarios Distribuidos	`redisRateLimiter` (Redis Cluster 7.2 para Token Bucket) + `pgBouncerPooler` (Transaction Pooling).	Requiere igualmente un componente externo (Redis/Envoy) para Rate Limiting por tenant.

D. Arquitectura Distribuida, Escalabilidad y Comparación de Modelos Multi-Tenant (Requisitos 1 y 2)

(Diagramas detallados en `docs/ArchitectureAndDataModel.md`)

- **Comparación Pool vs. Bridge vs. Silo:**

- **Base Compartida (Pool):** Usada para los **4.850 clientes** `FREE` **y** `PRO` **(97 %)** en `nodeAz1Pool` y `nodeAz2Pool`, garantizando bajo costo y alta velocidad de onboarding con aislamiento RLS.
- **Esquemas Separados (Bridge): Descartado** porque 5.000 esquemas x 7 tablas generan 35.000 tablas y >150.000 índices, degradando el catálogo `pg_class` y haciendo lentas las migraciones.
- **Bases / Nodos Separados (Silo):** Usado selectivamente para los **150 clientes** `ENTERPRISE` **(3 %)** aislando su shard en `nodeAz3Silo` (`TEST 06`).
- **Por qué aumentar contenedores NO equivale a escalar la base de datos:**
- Escalar los pods de `saasTenantApi` mediante `SaasApiAndAutoscaler.yaml` sólo añade cómputo HTTP *stateless*. Escalar la base de datos `saasCloudDb` requiere: (a) multiplexar conexiones en `pgBouncerPooler`, (b) repartir lecturas/escrituras entre shards distribuidos en `nodeAz1Pool`, `nodeAz2Pool` y `nodeAz3Silo`, y (c) aislar *Tenants Ballena* o añadir nuevos pods al `StatefulSet` `DatabaseCluster.yaml` ejecutando rebalanceo de shards.
- **Coordinación de Almacenamiento Persistente y Reemplazo de Nodos:**
- El `StatefulSet` utiliza la `StorageClass` `fast-ssd-multiaz-encrypted` (`StorageClass.yaml`) con `WaitForFirstConsumer`, `reclaimPolicy: Retain` y `allowVolumeExpansion: true`, junto con `topologySpreadConstraints` en 3 Zonas de Disponibilidad y `PodDisruptionBudget` (`minAvailable: 2`).

E. Modelo de Datos Explícito y Control de Redundancia (Requisito 3)

(Implementado en ***`sql/DataModel.sql`*** *)*

- **Tablas del modelo en camelCase:**
- `saasPlans` (*Reference Table* replicada en todos los nodos).
- `tenants`, `tenantUsers`, `projects`, `workOrders`, `invoices`, `auditEvents` (co-localizadas por `tenantId`).
- **Control de redundancia:** El modelo mantiene tercera forma normal (3FN) sin duplicar datos de negocio entre tablas, aprovechando que los `JOIN` co-localizados por `tenantId` se resuelven en el mismo shard en **`4,22 ms`**. La replicación síncrona (`RF=2` entre zonas) se usa exclusivamente para disponibilidad y tolerancia a fallos.

F. Estrategia de Particionamiento, Replicación, Fallos y Consistencia (Requisito 6)

- **Particionamiento:** Hash Sharding por `tenantId` (`computeShardId`).
- **Replicación Multi-AZ:** Cada shard primario en `nodeAz1Pool` tiene su réplica síncrona en `nodeAz2Pool` y viceversa.
- **Comportamiento ante Fallos y Particiones de Red (Teorema CAP):**
- No se promete disponibilidad y consistencia fuerte simultánea en ambos lados de una partición de red.
- La partición minoritaria rechaza escrituras para impedir *Split-Brain*, y al recuperarse aplica *STONITH fencing* limpiando datos primarios obsoletos antes de resincronizarse como réplica (`recoverFailedNode` en **`ClusterEngine.py`**, demostrado en `TEST 07`).

G. Reglas de Integridad, Idempotencia y Control de Concurrencia (Requisito 4)

- **Claves primarias y foráneas compuestas:** `PRIMARY KEY ("tenantId", "orderId")`, `FOREIGN KEY ("tenantId", "projectId")`.

- **Idempotencia:** `UNIQUE ("tenantId", "idempotencyKey")` en `workOrders` e `invoices` (`TEST 04`).
- **Control de Concurrencia Optimista (OCC):** Columna `"version" INT NOT NULL DEFAULT 1` verificada en `updateOrderStatusOcc` (`TEST 04` : 1 actualización exitosa y 9 conflictos HTTP 409 detectados sin pérdida de actualizaciones).

H. Implementación Mínima Reproducible (Entregable 3)

- **Configuración de clústeres, Kubernetes y Terraform:** `docker/docker-compose.yml` , `kubernetes/` , `terraform/` .
- **Scripts SQL en camelCase:** `sql/DataModel.sql` , `sql/ShardingAndDistribution.sql` , `sql/TenantSecurityRls.sql` , `sql/BusinessQueries.sql` , `sql/SchemaMigration.sql` .
- **Código Python ejecutable en camelCase:** `scripts/ClusterEngine.py` , `scripts/SyntheticData.py` (y `scripts/SynthticData.py`), `scripts/BusinessOperations.py` , `scripts/TestSuite.py` .
- **Datos sintéticos generados:** `data/TenantsCatalog5000.csv` , `data/Tenants.csv` , `data/Users.csv` , `data/Projects.csv` , `data/WorkOrders.csv` , `data/Invoices.csv` , `data/DatasetSummary.json` .

I, J, K. Pruebas, Plan de Responsabilidades y Costos

- **I. Pruebas y Resultados:** `docs/TestResultsReport.md` y `evidences/TestReport.json` .
- **J & K. Seguridad, Recuperación, Roles y Costos:** `docs/SecurityRecoveryAndCosts.md` .
- **Guía de Sustentación:** `docs/PresentationGuide.md` .

Diagrama 1: Arquitectura Distribuida Cloud Native Multi-AZ (saasCloudDb)

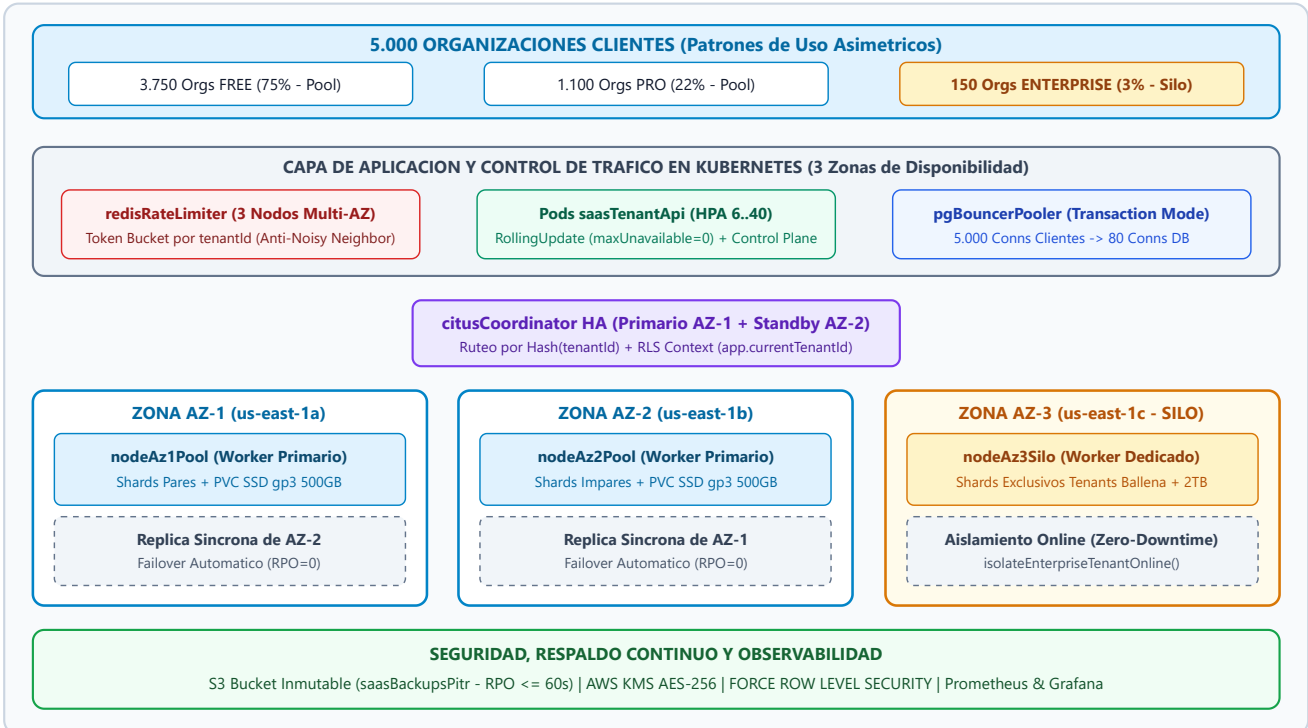


Diagrama 2: Modelo Entidad-Relacion Explicito Co-localizado por tenantId (saasCloudDb)

